Appendix A: Source Code and UNIX Orchestration Scripts

YanzhiBao

The core orchestration of the NOVA pipeline is implemented primarily utilizing native UNIX Bash scripting rather than high-level interpreted languages (e.g., Python) or containerized middleware. This deliberate architectural choice strictly aligns with the bare-metal optimization philosophy detailed in this dissertation. By utilizing Bash, the pipeline interfaces directly with the Linux kernel, enabling zero-copy data routing via standard I/O streams and facilitating hardware-level memory management (such as manually purging the OS page cache to prevent out-of-memory faults). This methodology entirely eliminates the Inter-Process Communication (IPC) and serialization overhead inherent in RESTful APIs, ensuring that the limited CPU threads are exclusively dedicated to AVX2 tensor computation. The following scripts are included directly within this document to provide transparent, reproducible evidence of these low-level systems engineering approach.

1.Linux Edge Node Environment and Dependency Installation Instructions

```
# 1. Update system and install core dependencies
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y build-essential cmake git curl wget jq tmux
sudo apt-get install -y python3-pip python3-venv ffmpeg alsa-utils libasound2-dev
sudo apt-get install -y docker.io docker-compose

# 2. Setup ALSA for Jabra Speak 410
# Check audio device mapping
aplay -l
arecord -l
sudo usermod -aG audio $USER

# 3. Build whisper.cpp (ASR)
git clone https://github.com/ggerganov/whisper.cpp.git
cd whisper.cpp
make -j4
bash ./models/download-ggml-model.sh base.en-q4_0
cd ..

# 4. Build llama.cpp (LLM)
git clone https://github.com/ggerganov/llama.cpp.git
cd llama.cpp
make -j4
mkdir models
wget https://huggingface.co/TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF/resolve/main/tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf -O models/tinyllama.gguf
cd ..
```

5. Install Piper (TTS)

wget https://github.com/rhasspy/piper/releases/download/v1.2.0/piper_linux_x86_64.tar.gz

tar -xf piper_linux_x86_64.tar.gz

2. Microservices Baseline Architecture (Before Optimization / Causes OOM)

```
# baseline_microservice.py

import requests

import json

import time

import subprocess

import os


ASR_API = "http://localhost:5001/transcribe"

LLM_API = "http://localhost:5002/generate"

AUDIO_TMP = "/tmp/input.wav"


def record_audio(duration=3):

    # record from hw:0,0 (Jabra)

    subprocess.run(["arecord", "-D", "hw:0,0", "-d", str(duration), "-f", "S16_LE", "-r", "16000",
AUDIO_TMP], check=True)


def run_pipeline():

    print("[Baseline] Starting REST pipeline...")

    try:

        # Step 1: ASR via HTTP Post

        with open(AUDIO_TMP, "rb") as f:

            res_asr = requests.post(ASR_API, files={"audio": f})

        text = res_asr.json().get("text", "")

        if not text: return


        # Step 2: LLM via HTTP JSON

        payload = {"prompt": text, "max_tokens": 50}

        res_llm = requests.post(LLM_API, json=payload, headers={"Content-Type":
```

```
"application/json"})
            reply = res_llm.json().get("response", "")

            print(f"AI: {reply}")

        except Exception as e:
            print(f"Pipeline crashed: {e}")

if __name__ == "__main__":
    while True:
        record_audio()
        run_pipeline()
        time.sleep(1)
```

3. Part 3: NOVA Bare-Metal Pipeline Architecture (Optimized / The Proposed Work)

```
#!/bin/bash

# nova_pipeline.sh

# Bare-metal offline voice assistant


set -e


VAD_SCRIPT="vad_alsa.py"

WAV_PATH="/tmp/nova_buffer.wav"

TXT_PATH="/tmp/nova_out.txt"


ASR_BIN="./whisper.cpp/main"

ASR_MOD="./whisper.cpp/models/ggml-base.en-q4_0.bin"


LLM_BIN="./llama.cpp/llama-cli"

LLM_MOD="./llama.cpp/models/tinyllama.gguf"


while true; do

    # 1. VAD Gate (blocks until speech)

    python3 $VAD_SCRIPT --out $WAV_PATH


    if [ ! -f "$WAV_PATH" ]; then continue; fi


    # 2. ASR Decoding

    TRANSCRIPT=$($ASR_BIN -m $ASR_MOD -f $WAV_PATH -nt | grep -v '\[')


    if [ -z "$TRANSCRIPT" ]; then continue; fi

    echo "U: $TRANSCRIPT"
```

```
    # 3. Memory flush before LLM (Crucial for 8GB RAM)
    sync
    echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null

    # 4. LLM Generation via mmap
    $LLM_BIN -m $LLM_MOD -n 64 --temp 0.3 -c 512 --use-mmap -p "$TRANSCRIPT" > $TXT_PATH

    # 5. TTS Playback
    cat $TXT_PATH | ./piper/piper --model en_US-lessac-medium.onnx --output_raw | aplay -r 22050 -f S16_LE -t raw

    rm -f $WAV_PATH $TXT_PATH
done
```

4. model_quantization_pipeline.sh

```
# This script must be run on a machine with sufficient RAM before transferring

# the compressed binaries to the target i5-6400T edge node.


set -e

LLAMA_DIR="./llama.cpp"

MODEL_NAME="TinyLlama-1.1B-Chat-v1.0"

RAW_DIR="./models/raw_fp16"

OUT_DIR="./models/quantized"

echo "[QUANT] Starting INT4 Quantization Pipeline..."

mkdir -p $OUT_DIR


# Step 1: Convert raw HuggingFace format to GGML FP16

echo "[QUANT] Converting PyTorch tensors to GGUF FP16..."

python3 $LLAMA_DIR/convert.py $RAW_DIR/$MODEL_NAME \

        --outfile $OUT_DIR/$MODEL_NAME-F16.gguf \

        --outtype f16


# Step 2: Apply K-quants (Q4_K_M provides optimal balance of perplexity and VRAM) echo
"[QUANT] Applying Q4_K_M quantization..."

$LLAMA_DIR/quantize \

$OUT_DIR/$MODEL_NAME-F16.gguf \

$OUT_DIR/$MODEL_NAME-Q4_K_M.gguf \ q4_k_m

echo "[SUCCESS] INT4 model ready for bare-metal deployment."
```

5.thermal_throttle_monitor.sh

```
LOG="thermal_metrics.csv"
echo "Timestamp,Core_Freq_MHz,Temp_Celsius" > $LOG

echo "Monitoring CPU Thermals. Press [CTRL+C] to stop."

while true; do
    TIME=$(date +"%H:%M:%S")

    # Extract current CPU frequency (Core 0 as baseline)
    FREQ=$(cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq)
    FREQ_MHZ=$((FREQ / 1000))

    # Extract package temperature
    TEMP=$(cat /sys/class/thermal/thermal_zone0/temp 2>/dev/null || echo "0")
    TEMP_C=$((TEMP / 1000))

    # Visual warning if hitting typical T-junction threshold for i5
    if [ "$TEMP_C" -gt 85 ]; then
        echo "[$TIME] WARNING: High Thermal Load ($TEMP_C C) - Potential Throttling"
    fi

    echo "$TIME,$FREQ_MHZ,$TEMP_C" >> $LOG
    sleep 2
done
```